操作系统

CPU 调度

周炽金

华东师范大学
软件工程学院

本章内容

线程已经可以独立执行；当就绪线程多于可用 CPU 时，系统还需要决定执行顺序。

- 调度在什么时候发生，调度器和分派程序各做什么？
- 如何用等待时间、周转时间和响应时间比较调度结果？
- FCFS、SJF、RR、优先级和多级反馈队列怎样选择任务？
- 多核、实时要求和实际操作系统带来哪些变化？
- 怎样用计算、模型、仿真和实际测量评价算法？

资料：1. Operating System Concepts，第 5 章。

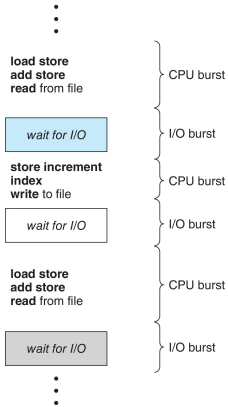
CPU 调度与就绪队列

- **CPU 调度（CPU scheduling）**：从就绪任务中选择下一个使用 CPU 的任务。
- **就绪队列（Ready queue）**：保存已经具备运行条件、正在等待 CPU 的任务。
- 任务等待 I/O 时不在就绪队列中；I/O 完成后重新就绪，再参与调度。

资料：1. Operating System Concepts，第 5 章。

CPU 执行与 I/O 等待交替出现

进程会计算，也会等待；当前进程等待 I/O 时，CPU 可以执行其他就绪进程。

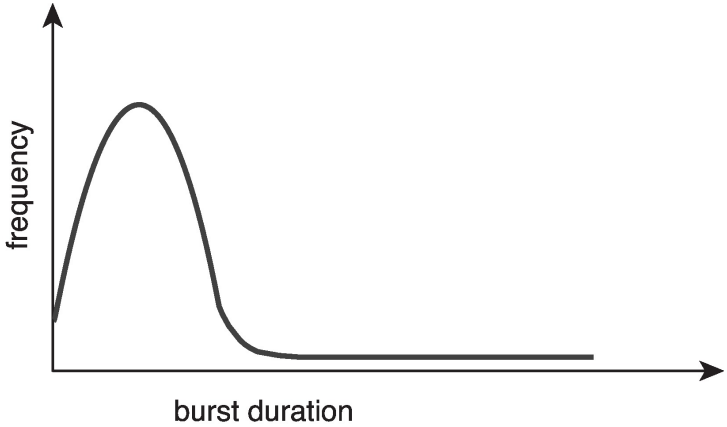


- CPU 执行区间（CPU burst）：连续执行计算的阶段。
- I/O 等待区间（I/O burst）：等待输入输出操作完成。
- 调度关心的是下一段 CPU 执行，而不是整个程序的全部运行时间。

资料：1. Operating System Concepts，第 5 章。

CPU 执行区间的长度分布

许多任务只需短时间计算就再次等待，少数任务会长时间占用 CPU。



资料：1. Operating System Concepts，第 5 章。

调度发生的时机

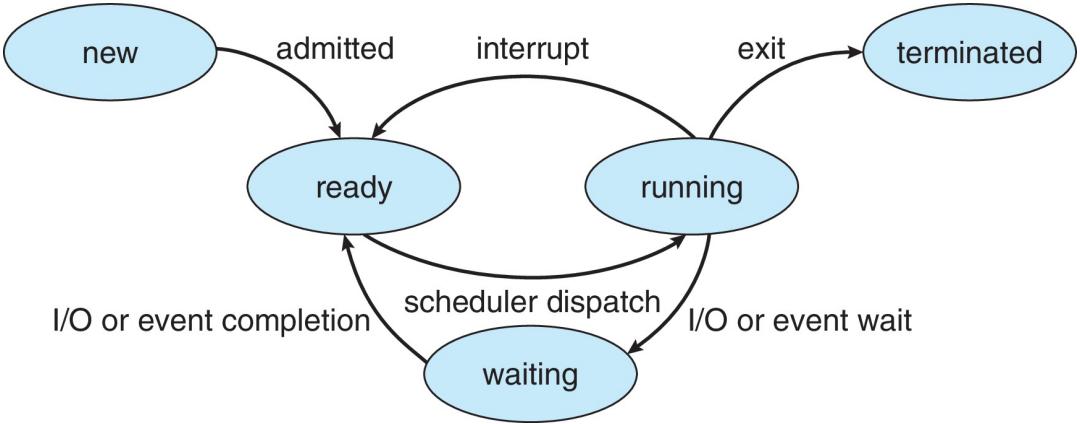
当前任务阻塞或退出时，CPU 需要交给其他任务；新任务就绪时，也可能重新调度。

状态变化	当前任务能否继续运行	调度含义
运行 → 等待	不能	有其他就绪任务时选择一个执行
运行 → 就绪	已被换下	例如时间片用完，重新排队
等待 → 就绪	原任务仍可继续运行	新就绪任务可能触发抢占
运行 → 退出	不能	选择其他任务，或进入空闲

资料：1. Operating System Concepts，第 5 章。

调度时机与进程状态

状态图中的 running → ready 对应抢占，waiting → ready 对应 I/O 等待结束。



资料：1. Operating System Concepts，第 5 章。

抢占式与非抢占式调度

当前任务仍可运行时，操作系统是否允许把 CPU 分配给另一个任务?

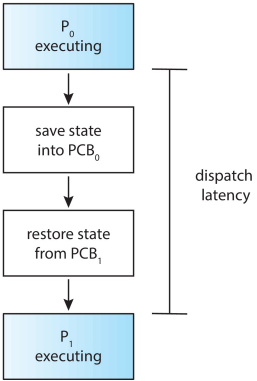
方式	当前任务何时让出 CPU	典型例子
非抢占式（Nonpreemptive）	阻塞或执行结束	本章的 FCFS、非抢占 SJF
抢占式（Preemptive）	也可在时间片用完或更合适任务就绪时换下	RR、SRTF、抢占优先级

- 抢占提高响应机会，但会增加切换开销。
- 任务可能在更新共享数据时被抢占，因此访问共享数据需要同步保护。具体机制将在进程同步一章讨论。
- Linux、Windows 等通用系统支持抢占调度。

资料：1. Operating System Concepts，第 5 章；2. sched(7)；9. Windows：调度优先级。

调度器与分派程序

调度器（CPU scheduler）选择任务，分派程序（Dispatcher）完成切换，让任务开始或恢复执行。



- 保存原任务现场，恢复新任务现场。
- 必要时切换地址空间，并恢复适当的执行模式。
- 跳转到任务应继续执行的位置。
- 分派延迟（Dispatch latency）是停止原任务到开始新任务所需的时间。

资料：1. Operating System Concepts，第 5 章。

调度的评价指标

调度顺序会影响 CPU 的忙碌程度，以及每个任务等待和完成的时间。

指标	含义	一般目标
CPU 利用率（CPU utilization）	CPU 忙碌时间占观察时间的比例	提高
吞吐量（Throughput）	单位时间完成的任务数	提高
周转时间（Turnaround time）	任务到达到完成的时间	缩短
等待时间（Waiting time）	在就绪队列中等待的总时间	缩短
响应时间（Response time）	到达到首次获得服务的时间	缩短

资料：1. Operating System Concepts，第 5 章。

周转、等待与响应时间的计算

三个时间指标的起止点不同；响应时间只看第一次服务，等待时间要累计所有排队阶段。

设到达时刻为 A，首次运行时刻为 S，完成时刻为 C。

$$T_{turn} = C - A, \qquad T_{response} = S - A$$

等待时间是在就绪队列中各次等待时间的总和。设每次等待的时长为 $w_1, w_2, \ldots, w_n$：

$$T_{wait} = w_1 + w_2 + \cdots + w_n$$

- 既包括首次运行前的等待，也包括运行过程中重新进入就绪队列后的等待。
- 平均指标等于各任务对应指标之和除以任务数。

资料：1. Operating System Concepts，第 5 章。

一次分段执行的三个时间指标

任务第一次运行后仍可能重新排队，因此等待时间不一定等于响应时间。



任务在时刻 1 到达，在 3–5 和 8–10 运行，其余标记区间由其他任务执行。

- 响应时间：$3 - 1 = 2$。
- 周转时间：$10 - 1 = 9$；CPU 执行时间为 4。
- 等待时间：$(3 - 1) + (8 - 5) = 5$。

资料：1. Operating System Concepts，第 5 章。

不同指标之间的取舍

平均完成得快、每个任务都能及时响应，以及切换开销小，往往不能同时最优。

- 批处理常关心吞吐量、平均周转时间和资源利用率。
- 交互程序更关心首次响应，以及较长等待是否经常出现。
- 实时任务还要求在截止时间之前完成。
- 只报告平均值，可能掩盖少数任务的长时间等待。
- 算法选择取决于系统更重视吞吐量、响应速度，还是截止时间。

资料：1. Operating System Concepts，第 5 章。

基本调度算法

下面比较四种选择规则：先到先执行、短任务优先、时间片轮转和优先级调度。

- FCFS：按进入就绪队列的顺序执行。
- SJF / SRTF：优先执行短任务或剩余时间短的任务。
- RR：每个任务轮流使用一个时间片。
- 优先级调度：先选择优先级高的任务。

资料：1. Operating System Concepts，第 5 章。

先来先服务（FCFS）

先来先服务（First-Come, First-Served，FCFS）按进入就绪队列的顺序执行。

- 新就绪任务进入队尾，CPU 空闲时选队首。
- 当前任务一直运行到本次 CPU 执行结束或发生阻塞。
- 规则简单，但等待时间对任务的排队顺序很敏感。
- 长任务先执行时，后面的短任务也必须等待。

资料：1. Operating System Concepts，第 5 章。

FCFS：长任务排在前面

P1、P2、P3 在时刻 0 到达，按此顺序排队；执行期间没有 I/O，忽略切换开销。

进程	P1	P2	P3
CPU 执行时间	24	3	3



- P1 从时刻 0 执行到 24，随后 P2、P3 依次运行。
- 等待时间分别为 0、24、27。
- 平均等待时间：$(0 + 24 + 27)/3 = 17$。

资料：1. Operating System Concepts，第 5 章。

FCFS：只改变初始排队顺序

保持执行时间和到达时刻相同，只把初始队列改为 P2、P3、P1。



- P2、P3、P1 的等待时间为 0、3、6。
- 平均等待时间：$(0+3+6)/3=3$。
- 执行总量仍为 30，但短任务不再被长任务挡在后面。

资料：1. Operating System Concepts，第 5 章。

护航效应（Convoy effect）

一个长任务挡住多个短任务，就像一辆慢车后面排起了车队。

- CPU 密集型（CPU-bound）任务占用 CPU 时，多个 I/O 密集型（I/O-bound）任务在就绪队列中等待。
- 短任务得不到 CPU，就不能及时发起下一次 I/O。
- 长任务开始等待 I/O 后，短任务才陆续运行，又很快一起等待 I/O。
- CPU 与 I/O 设备可能交替空闲，整体资源利用受影响。

资料：1. Operating System Concepts，第 5 章。

短作业优先（SJF）

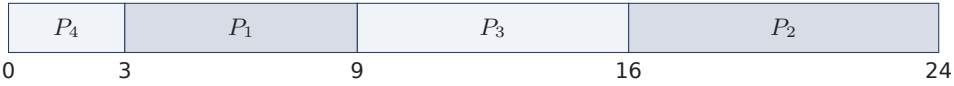
短作业优先（Shortest-Job-First，SJF）优先运行下一段 CPU 执行时间较短的任务。

- **非抢占式 SJF**：每次选择下一段 CPU 执行时间最短的就绪任务。
- 任务开始运行后，不因新短任务到达而被抢占。
- 当一组任务同时到达、执行时间已知且忽略切换开销时，SJF 的平均等待时间最小。
- 实际困难：下一段 CPU 执行时间通常只能估计。

资料：1. Operating System Concepts，第 5 章。

SJF 的执行顺序与等待时间

四个任务同时到达，执行时间分别为 P1=6、P2=8、P3=7、P4=3。



进程	P1	P2	P3	P4
等待时间	3	16	9	0

平均等待时间：$(3+16+9+0)/4=7$。

资料：1. Operating System Concepts，第 5 章。

为什么短任务优先能减少平均等待?

只比较相邻的两个任务，就能说明同一时刻到达时为什么应该短任务在前。

设两个任务从时刻 t 起执行，长度分别为 a、b，且 $a > b$。

执行顺序	两个任务的等待时间之和
长任务 a → 短任务 b	$t + (t + a) = 2t + a$
短任务 b → 长任务 a	$t + (t + b) = 2t + b$

- 交换后，等待时间之和减少 $a - b$。
- 后续任务的开始时刻不变；重复交换即可得到按长度递增的顺序。

资料：1. Operating System Concepts，第 5 章。

最短剩余时间优先（SRTF）

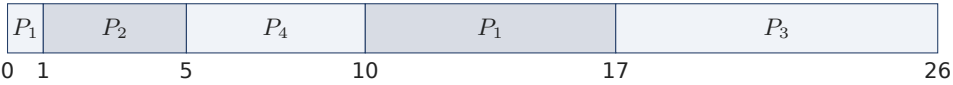
最短剩余时间优先（Shortest-Remaining-Time-First，SRTF）是 SJF 的抢占式版本。

- 就绪任务中，剩余 CPU 执行时间最短者优先。
- 新到达任务若比当前任务的剩余时间更短，就发生抢占。
- 比较当前任务的剩余时间与新任务的执行时间，决定是否抢占。
- 需要估计执行长度，也可能让长任务长期等待。

资料：1. Operating System Concepts，第 5 章。

SRTF：有新任务到达时重新比较

P1–P4 的到达时刻为 0、1、2、3，执行时间为 8、4、9、5；同长时先到者优先。



- 时刻 1：P1 剩余 7，P2 只需 4，因此 P2 抢占。
- 时刻 2、3：P2 的剩余时间仍最短，继续运行到时刻 5。
- 随后依次完成 P4、P1、P3；平均等待时间为 6.5。

资料：1. Operating System Concepts，第 5 章。

CPU 执行时间的预测

SJF 需要估计下一段执行时间，可以使用指数平均（Exponential averaging）结合历史记录预测。

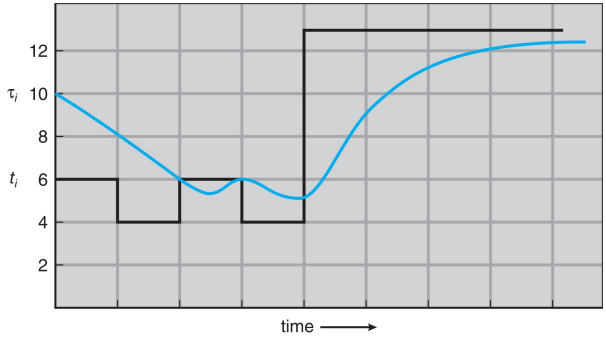
$$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n, \quad 0 \le \alpha \le 1$$

- t_n：第 n 次 CPU 执行区间的实际长度。
- τ_n：对第 n 次的预测；τ_{n+1} 用于预测下一次。
- 例如 $\alpha = 0.5$、$\tau_n = 10$、$t_n = 6$，则下一次预测为 8。
- 预测结果用于选择任务，实际执行后再更新下一次预测。

资料：1. Operating System Concepts，第 5 章。

指数平均：预测如何随实际值变化

黑线为实际执行时间，蓝线为预测值；本例取初始预测 10、权重 α=0.5。



CPU burst (t_i)	6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

资料：1. Operating System Concepts，第 5 章。

指数平均：新旧数据的权重

α 越大，最近一次执行时间对下一次预测的影响越大。

- $\alpha = 0$：预测始终保持原值，新数据不起作用。
- $\alpha = 1$：只采用最近一次的实际长度。
- $0 < \alpha < 1$：越早的实际观测，权重按几何比例减小。

$$\tau_{n+1} = \alpha \sum_{j=0}^{n} (1-\alpha)^j t_{n-j} + (1-\alpha)^{n+1} \tau_0$$

较大的 alpha 反应快，也更容易随短期波动变化。

资料：1. Operating System Concepts，第 5 章。

时间片轮转（RR）

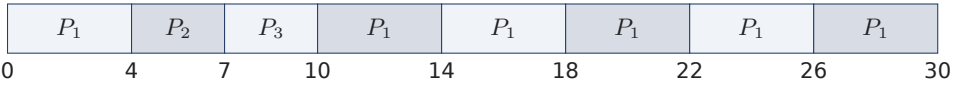
时间片轮转（Round Robin，RR）给每个就绪任务一个时间片（Time quantum），轮流运行。

- 队首任务最多运行一个时间片 q。
- 提前完成或阻塞，就提前让出 CPU。
- 时间片用完但还可运行，则回到就绪队列尾部。
- 同优先级的就绪任务按队列顺序轮流运行。

资料：1. Operating System Concepts，第 5 章。

RR：时间片 q=4

沿用 P1=24、P2=3、P3=3，全部在时刻 0 到达；按 P1、P2、P3 排队，q=4。



- P1 首次只运行 4；P2、P3 分别在时刻 4、7 获得 CPU。
- P2、P3 完成后只剩 P1，后续时间片连续用于 P1。

资料：1. Operating System Concepts，第 5 章。

RR 中的响应时间与等待时间

轮流运行可以让任务早些获得 CPU，但首次响应快不等于等待总量小。

指标	P1	P2	P3
首次运行时刻	0	4	7
完成时刻	30	7	10
响应时间	0	4	7
等待时间	6	4	7

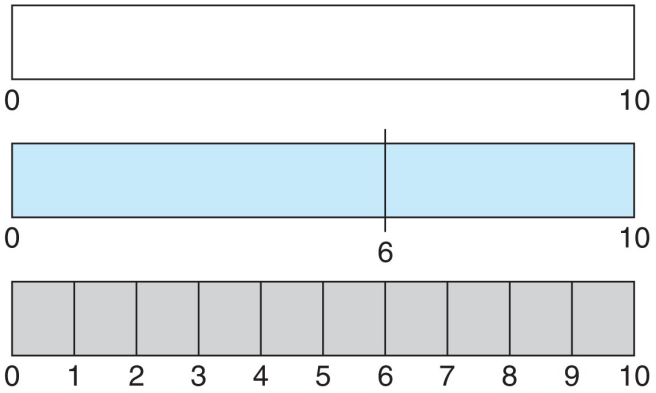
平均响应时间为 $11/3$，平均等待时间为 $17/3$。

固定 n 个持续就绪任务时，不计切换，轮到下一次执行前最多等待 $(n-1)q$。

资料：1. Operating System Concepts，第 5 章。

时间片与切换次数

时间片越短，调度机会越频繁；切换成本占比也可能增加。

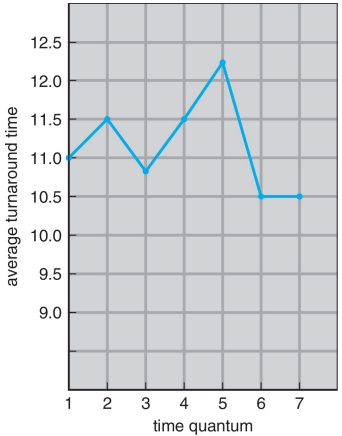


quantum	context switches
12	0
6	1
1	9

资料：1. Operating System Concepts，第 5 章。

时间片长度对周转时间的影响

时间片改变了任务交错方式，平均周转时间不一定随 q 单调变化。



process	time
P_1	6
P_2	3
P_3	1
P_4	7

资料：1. Operating System Concepts，第 5 章。

优先级调度

优先级调度（Priority scheduling）允许重要任务优先获得 CPU。

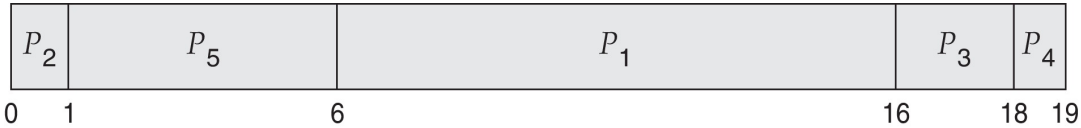
- 从就绪任务中选择优先级最高者；可以采用抢占式或非抢占式。
- 优先级相同，还需要 FCFS 或 RR 等规则决定先后。
- 下面的例题规定：**数字越小，优先级越高**。
- 若把预测执行时间越短的任务设为越高优先级，就得到 SJF 的选择规则。

资料：1. Operating System Concepts，第 5 章。

非抢占优先级调度示例

所有任务同时到达，数字越小优先级越高；同级按题目顺序执行。

进程	P1	P2	P3	P4	P5
CPU 时间	10	1	2	1	5
优先级	3	1	4	5	2



平均等待时间：$(6+0+16+18+1)/5 = 8.2$。

资料：1. Operating System Concepts，第 5 章。

饥饿与老化

高优先级任务不断到来时，低优先级任务可能一直得不到 CPU。

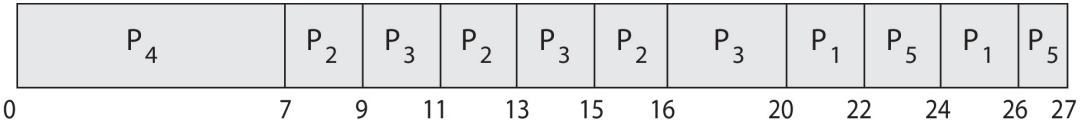
- **饥饿（Starvation）**：任务已就绪，却长期没有执行机会。
- **老化（Aging）**：等待越久，逐步提高其优先级。
- 数值越小表示越高时，提高优先级需要减小数值。
- 也可以周期性提升等待队列，或为低优先级任务预留份额。

资料：1. Operating System Concepts，第 5 章。

优先级调度与时间片轮转

优先级与 RR 可以组合：先选最高优先级，再让同级任务轮流运行。

进程	P1	P2	P3	P4	P5
CPU 时间	4	5	8	7	3
优先级	3	2	2	1	3



数字越小优先级越高，q=2；先完成 P4，再轮转 P2/P3，最后轮转 P1/P5。

资料：1. Operating System Concepts，第 5 章。

课堂练习：比较四种算法

P1–P5 按题目顺序在时刻 0 到达，执行时间和优先级如下。

假设有如下一组进程，它们的 CPU 执行时间以毫秒来计算：

进程	执行时间	优先级
P_1	2	2
P_2	1	1
P_3	8	4
P_4	4	2
P_5	5	3

假设进程按 P_1、P_2、P_3、P_4、P_5 顺序在时刻 0 到达。

考虑以下四种调度算法：FCFS、非抢占 SJF、非抢占优先级（数字越大优先级越高）、RR（时间片为 2）。对于上述这组进程而言，每个调度算法的平均等待时间分别是多少？

同优先级按题目顺序执行；忽略切换开销。

几种基本算法的比较

FCFS、SJF、RR 和优先级调度，分别依据到达顺序、执行长度、轮转机会和任务优先级选择。

算法	依据	主要问题
FCFS	就绪队列顺序	长任务可能阻挡短任务
SJF / SRTF	执行长度或剩余长度	长度难预测，长任务可能饥饿
RR	时间片与队列顺序	响应与切换开销需要权衡
优先级	任务的优先级	同级规则、饥饿与老化

选择规则不同，就会产生不同的执行顺序和等待时间。

资料：1. Operating System Concepts，第 5 章。

多级队列：不同任务采用不同规则

系统既有交互任务，也有后台计算任务，未必适合全部使用同一条队列。

- 先按优先级或任务类型，将就绪任务分入不同队列。
- 每个队列可以使用自己的算法，如 RR 或 FCFS。
- 队列之间还要决定先服务哪一个，或怎样分配 CPU 份额。
- 如果任务特征会变化，还需要允许任务在队列间移动。

先看分类固定的多级队列，再看允许任务换队的多级反馈队列。

资料：1. Operating System Concepts，第 5 章。

多级队列调度（MLQ）

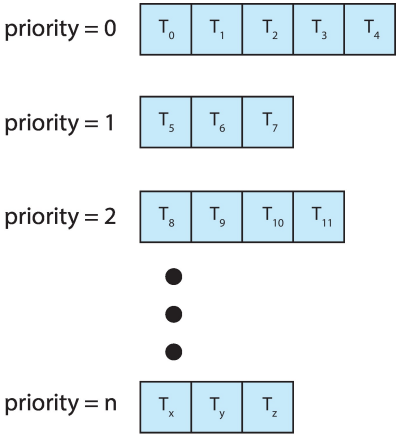
多级队列（Multilevel queue，MLQ）把就绪任务分类，各队列分别采用自己的调度规则。

- **队列数量**：需要划分多少个服务级别？
- **入队规则**：按优先级、前台后台或任务类型进入哪一队？
- **队内算法**：各队列分别使用 FCFS、RR 或其他规则。
- **队间规则**：严格优先级，或按比例分配 CPU 时间。
- 任务按固定类别入队，不根据运行行为自动换队。

资料：1. Operating System Concepts，第 5 章。

按优先级划分多个队列

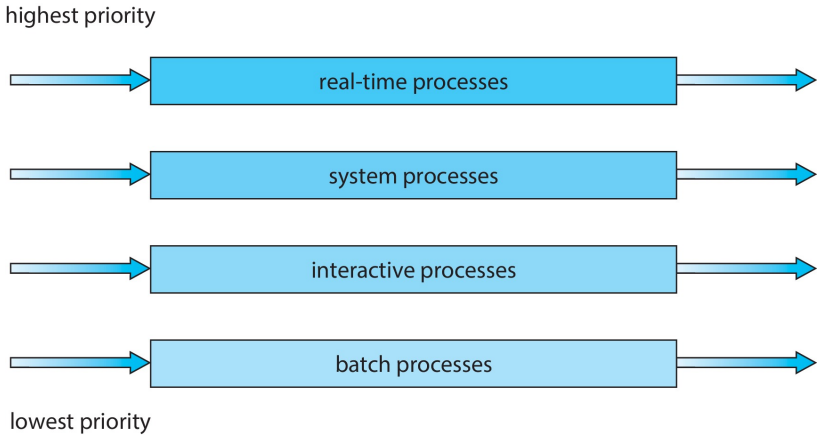
采用严格优先级时，总是先从最高的非空队列中选择任务。



资料：1. Operating System Concepts，第 5 章。

按任务类型划分多个队列

同样的多队列结构，也可以按实时、系统、交互和批处理等类型分类。



资料：1. Operating System Concepts，第 5 章。

多级反馈队列（MLFQ）

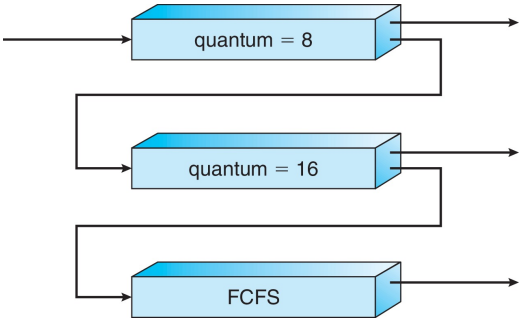
多级反馈队列（Multilevel feedback queue，MLFQ）根据任务的运行行为调整它所在的队列。

- 任务可以在队列之间移动，队列仍可采用不同时间片和算法。
- 需要规定初始队列、降级条件、升级条件和队间选择规则。
- 用完较短时间片的任务可以移到较低队列，减少它对短任务的影响。
- 老化或周期性提升用于改善低队列长期等待。
- 除降级外，还要规定提前阻塞后的处理方式，以及等待过久时的提升规则。

资料：1. Operating System Concepts，第 5 章。

三层反馈队列示例

新任务先进入 Q0，未完成时按已经用掉的 CPU 时间逐级下移。



- Q0：RR，时间片 8 ms。
- Q1：RR，时间片 16 ms。
- Q2：FCFS。
- 本例按队列严格优先：高队列就绪任务可以抢占低队列任务。

资料：1. Operating System Concepts，第 5 章。

反馈队列中的一次任务执行

一个需要 30 ms CPU 时间、始终不阻塞的任务，在没有其他任务时怎样移动?

阶段	本阶段执行	还剩多少	接下来进入
Q0	8 ms	22 ms	Q1
Q1	16 ms	6 ms	Q2
Q2	6 ms	0	退出

- 8 ms 和 16 ms 是先后获得的执行配额，第二层再运行 16 ms。
- 新的短任务可以先在 Q0 获得服务，长任务则逐步移到低队列。

资料：1. Operating System Concepts，第 5 章。

多核调度：选择任务与分配 CPU

多个 CPU 可以同时执行任务，调度器既要选择任务，也要决定任务放在哪个 CPU 上。

- 用户线程如何参与 CPU 竞争?
- 采用共享就绪队列，还是为各 CPU 分别建立队列?
- 怎样在 CPU 之间分配负载，减少任务迁移的代价?
- 操作系统看到的逻辑 CPU 与物理 CPU 核有什么区别?

下面依次讨论线程竞争范围、队列组织、负载均衡和硬件线程。

资料：1. Operating System Concepts，第 5 章。

用户线程与内核线程的竞争范围

线程竞争范围（Contention scope）规定了一个线程与哪些线程竞争 CPU。

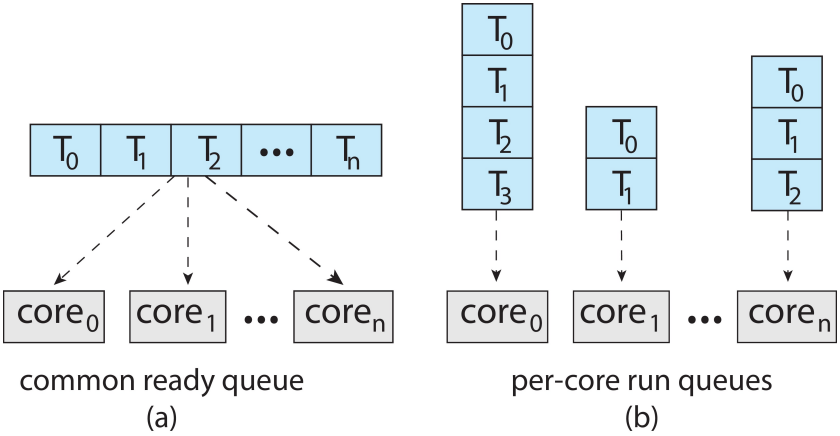
范围	竞争者	调度层次
进程竞争范围（Process contention scope，PCS）	同一进程内的相关用户线程	线程库或运行时管理
系统竞争范围（System contention scope，SCS）	同一调度域中的系统线程	内核调度

- POSIX 用 PTHREAD_SCOPE_PROCESS / SYSTEM 表达竞争范围。
- Linux NPTL 支持 SYSTEM，不支持 PROCESS；常规原生线程采用一对一实现。

资料：1. Operating System Concepts，第 5 章；3. 线程竞争范围。

共享就绪队列与独立运行队列

对称多处理（Symmetric multiprocessing，SMP）中，各 CPU 都参与调度。下图比较两种队列组织方式。



资料：1. Operating System Concepts，第 5 章。

多核调度中的负载均衡

负载均衡（Load balancing）把任务从繁忙 CPU 迁到较空闲的 CPU，减少一边排队、一边空闲的情况。

- 负载不仅是任务数量，还包括可运行时间、优先级和 CPU 能力。
- 可以由忙碌 CPU 迁出任务，或由空闲 CPU 从其他队列取任务。
- 迁移能改善空闲，但会改变缓存状态和数据访问位置。
- 迁移还受 CPU 亲和性（Processor affinity）与内存位置的影响，下一页分别说明。

资料：1. Operating System Concepts，第 5 章；4. CPU 亲和性。

CPU 亲和性与 NUMA

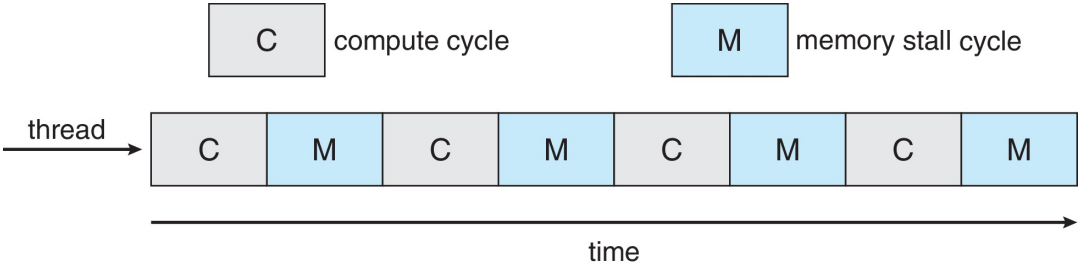
亲和性决定任务倾向或允许在哪个 CPU 上运行，NUMA 则影响它访问内存的代价。

- **软亲和性（Soft affinity）**：调度器尽量保留原 CPU，以利用已有缓存。
- **硬亲和性（Hard affinity）**：限制线程可执行的 CPU 集合，不保证独占这些 CPU。
- **非一致内存访问（Non-uniform memory access，NUMA）**：访问本地内存与其他节点内存的代价可能不同。
- 亲和性可减少迁移，却也可能让任务在繁忙 CPU 上排队。

资料：1. Operating System Concepts，第 5 章；4. CPU 亲和性。

访存等待对 CPU 执行的影响

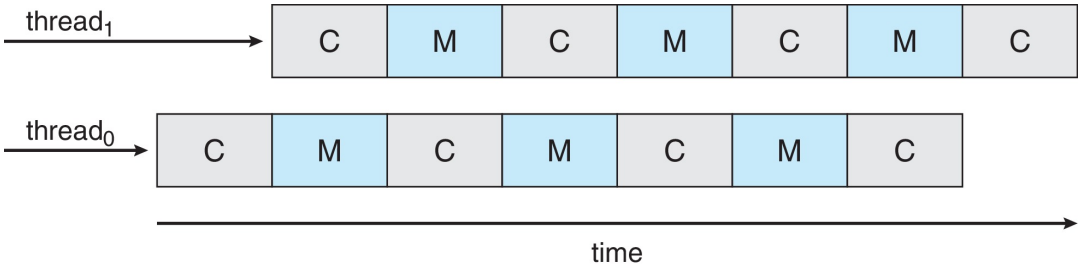
访存停顿（Memory stall）期间，CPU 正在等待内存数据，部分执行资源处于空闲状态。



资料：1. Operating System Concepts，第 5 章。

同一 CPU 核上的硬件线程

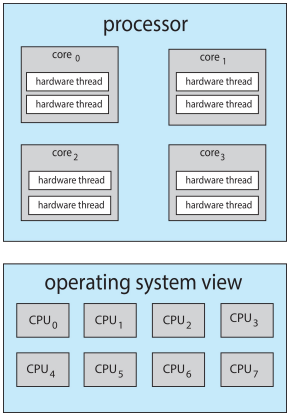
硬件多线程（Hardware multithreading）可以利用一个线程的访存停顿，执行另一线程的指令。



资料：1. Operating System Concepts，第 5 章。

物理 CPU 核与逻辑 CPU

支持每核两个硬件线程时，四个物理 CPU 核可以向操作系统呈现八个逻辑 CPU。



- 操作系统向逻辑 CPU 分配软件线程。
- 同核硬件线程共享部分执行资源和缓存。
- 8 个逻辑 CPU 不等于 8 个独立物理 CPU 核。
- 超线程（Hyper-Threading）是 Intel 对其相关技术的命名。

资料：1. Operating System Concepts，第 5 章。

实时调度与截止时间

实时调度（Real-time scheduling）关注任务能否在截止时间（Deadline）之前完成。

- **硬实时（Hard real-time）**：错过截止时间可能使系统不能满足功能要求。
- **软实时（Soft real-time）**：允许一定延迟，但需要控制错过期限造成的影响。
- 高优先级或平均响应快，都不能单独保证截止时间。
- 可行性依赖最坏执行时间、到达规律、阻塞和系统开销。

下面只讨论单 CPU 上可抢占、互相独立的周期任务模型。

资料：1. Operating System Concepts，第 5 章；15. Liu 与 Layland（1973）。

周期任务与调度参数

要判断能否按时完成，必须先把每个周期内的计算需求写清楚。

- C_i：一次任务的最坏执行时间（Worst-case execution time，WCET）。
- T_i：相邻两次任务释放的周期。
- D_i：从释放到截止时间的间隔；以下例子取 $D_i = T_i$。
- 单 CPU 利用率需求为 $U = \sum_i C_i/T_i$。

每个周期都会产生一次新的执行请求，都需要在各自的截止时间前完成。

资料：1. Operating System Concepts，第 5 章；15. Liu 与 Layland（1973）。

RM 与 EDF 的选择规则

速率单调调度（Rate-monotonic scheduling，RM）按周期确定优先级；最早截止时间优先（Earliest-deadline-first，EDF）按截止时间确定优先级。

算法	优先依据	优先级是否改变
速率单调调度（RM）	周期越短越优先	任务之间固定
最早截止时间优先（EDF）	当前作业的绝对截止时间越早越优先	随待执行作业变化

例：任务 A 为 C=1、T=4，B 为 C=2、T=5。在时刻 0，A 截止于 4，B 截止于 5，两种规则均先选 A。

资料：1. Operating System Concepts，第 5 章；15. Liu 与 Layland（1973）。

实时任务的可调度性

先计算任务的 CPU 利用率需求，再检查是否满足对应算法的可调度条件。

在独立、可抢占、无阻塞和开销、单 CPU、$D_i = T_i$ 的周期模型中：

- EDF 的可调度条件为 $U \leq 1$。
- n 个任务采用 RM 时，$U \leq n(2^{1/n} - 1)$ 是充分条件。
- 超过 RM 的充分界限，不等于一定无法调度。
- 前例 $U = 1/4 + 2/5 = 0.65$，低于两个任务的 RM 界限约 0.828。

资料：1. Operating System Concepts，第 5 章；15. Liu 与 Layland（1973）。

Linux 与 Windows 的调度方式

前面的算法在实际系统中会组合使用。下面看 Linux 的调度类和 Windows 的线程优先级。

系统	选择任务的主要方式	后续内容
Linux	区分普通、实时等调度类，类内采用各自的规则	公平分配、nice 与实时策略
Windows	先选择高优先级线程，同级采用时间片轮转	基础优先级与动态提升

先看 Linux 普通任务调度的演变，再比较两个系统中的优先级含义。

资料：1. Operating System Concepts，第 5 章；2. sched(7)；9. Windows：调度优先级。

Linux 调度算法的演变

Linux 的普通任务调度从优先级队列发展到公平分配，再改进任务选择方式。

时期	普通任务的主要设计
Linux 2.5 之前	传统 UNIX 风格的优先级与时间片调度
2.5 开发期 / 2.6 早期	O(1) 调度器，按优先级组织运行队列
2.6.23 起	CFS（Completely Fair Scheduler），按虚拟运行时间分配 CPU
6.6 起	公平调度开始采用 EEVDF

CFS 意为“完全公平调度器”；EEVDF 的选择规则在后面介绍。

资料：1. Operating System Concepts，第 5 章；5. CFS；6. EEVDF。

Linux 的调度策略与类别

调度类（Scheduling class）把普通任务、实时任务和截止时间任务交给不同的选择规则。

- 普通任务：SCHED_OTHER；还可有 SCHED_BATCH、SCHED_IDLE 等策略。
- 实时策略：SCHED_FIFO、SCHED_RR，使用实时优先级。
- 截止时间策略：SCHED_DEADLINE，指定执行预算、周期和期限。
- 常规配置下，截止时间、实时、普通类别按相应层次选择任务。
- 一个类别内部仍有自己的队列、计费和抢占规则。

资料：2. sched(7)；8. SCHED_DEADLINE。

Linux：nice 与 CPU 份额

nice 值影响普通任务的调度权重（Scheduling weight），进而影响它能获得的 CPU 时间份额。

- nice 范围为 **−20 到 19**；数值越小，普通任务的相对权重越高。
- 对同一公平调度组内持续就绪的任务，CPU 份额近似按权重分配。
- 相同 nice 的持续就绪任务，获得的长期份额接近。
- cgroup 配额与权重、亲和性及就绪状态都会影响实际份额。

资料：2. sched(7)；7. nice 权重；14. cgroup v2 CPU 控制。

从 CFS 的虚拟运行时间到 EEVDF

EEVDF（Earliest Eligible Virtual Deadline First）先判断任务是否应当获得服务，再比较虚拟截止时间。

- **CFS**：按权重累计虚拟运行时间，优先考虑虚拟运行时间较小者。
- **EEVDF**：先找尚未超用应得 CPU 份额的任务，再比较虚拟截止时间。
- 在符合执行条件的任务中，选择虚拟截止时间最早者，兼顾公平与延迟。
- 虚拟截止时间用于安排公平调度；实时任务的截止时间则来自应用需求。

资料：5. CFS；6. EEVDF。

Linux 优先级数值的不同含义

用户实时优先级、普通任务的 nice 和内核内部编号，分别表示不同的调度信息。

表示方式	取值与含义
用户接口的实时优先级	FIFO / RR 通常为 1–99，越大越高
普通任务的 nice	−20–19，越小权重越高
常见内核普通静态优先级	nice −20 对应 100，nice 19 对应 139
常见内核实时内部区间	低于 100 的内部编号，与用户实时数值方向不同

因此，用户实时优先级的“越大越高”，与内核编号的方向不同。

资料：2. sched(7)；16. Linux 内部优先级。

Windows 的线程优先级调度

Windows 选择就绪线程时，先考虑线程优先级，再安排同级线程获得 CPU。

- 线程优先级范围为 0–31，其中 0 用于特殊系统用途。
- 1–15 为可动态调整的范围；16–31 为实时优先级范围。
- 线程的基础优先级由进程优先级类别与线程相对优先级共同确定。
- 更高优先级线程就绪时，可以抢占较低优先级线程。
- 同优先级线程采用时间片轮转，时间片设置取决于系统。

资料：9. Windows：调度优先级。

Windows 的动态优先级调整

基础优先级之外，系统还会针对部分唤醒和交互场景作临时调整。

- 某些 I/O 完成或等待结束后，线程可获得临时优先级提升。
- 提升后的优先级会按规则回落到基础优先级。
- 实时范围的线程不采用这种动态提升。
- 动态提升（Priority boost）使刚结束部分等待操作的线程更快获得 CPU。

资料：9. Windows：调度优先级；10. Windows：动态提升。

在 Linux 中观察线程调度属性

先读取调度策略、nice 与 CPU 允许集合，再解释实际运行结果。

```
ps -L -p <PID> -o pid,tid,cls,rtprio,ni,psr,comm
chrt -p <TID>
taskset -pc <TID>
```

- PID、TID 分别换成进程与线程编号，不直接输入尖括号。
- NI 是 nice；RTPRIO 属于实时优先级；PSR 是最近执行的 CPU。
- taskset 查询允许执行的 CPU 集合，和最近执行位置是两回事。
- 以上命令为读取查询，不修改策略或亲和性。

资料：11. ps(1)；12. chrt(1)；13. taskset(1)。

算法评估：先确定输入和评价目标

最后比较四种算法评估方法：固定输入计算、排队模型、仿真和实际测量。

- **确定性评估（Deterministic modeling）**：给定同一组任务，直接计算各算法指标。
- **排队模型（Queueing model）**：用到达率、服务时间分布估计长期行为。
- **仿真（Simulation）**：重放或生成工作负载，统计执行结果。
- **实际系统测量**：观察完整实现的开销与性能。

先用同一组任务直接计算，再看持续到达的请求怎样建模。

资料：1. Operating System Concepts，第 5 章。

确定性评估：同一组任务作为输入

五个任务都在时刻 0 到达，使用原题顺序，忽略切换开销。

Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

- P1–P5 的执行时间为 10、29、3、7、12 ms。
- 比较 FCFS、非抢占 SJF、RR。
- RR 的时间片明确取 10 ms。
- 对每种算法计算同一组任务的平均等待时间。

资料：1. Operating System Concepts，第 5 章。

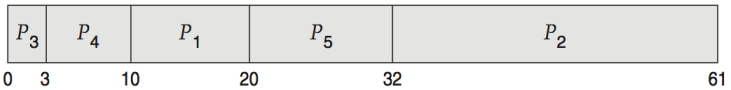
同一输入的三种调度结果

根据上一页的输入，三种算法的平均等待时间分别如下。

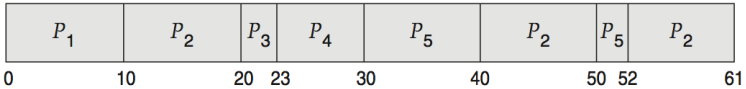
FCFS：28 ms



非抢占 SJF：13 ms



RR（q=10）：23 ms



这组任务同时到达，SJF 先完成短任务，平均等待时间最小。

资料：1. Operating System Concepts，第 5 章。

用排队模型描述长期负载

固定例题只处理一批任务；请求持续到达时，可以用排队模型描述到达和服务过程。

- 到达率（Arrival rate）λ 表示单位时间进入系统或队列的任务数。
- 服务率（Service rate）μ 描述服务能力，必须说明是单个服务台还是总能力。
- 在特定单服务台稳定模型中，利用率可写为 $\rho = \lambda/\mu$。
- CPU、I/O 与各自等待队列可以组成排队网络。
- 指定到达和服务分布后，估计利用率、队列长度与等待时间。

资料：1. Operating System Concepts，第 5 章。

Little 定律：队列长度与等待时间

Little 定律（Little’s law）把平均队列长度、到达率和平均等待时间联系起来。

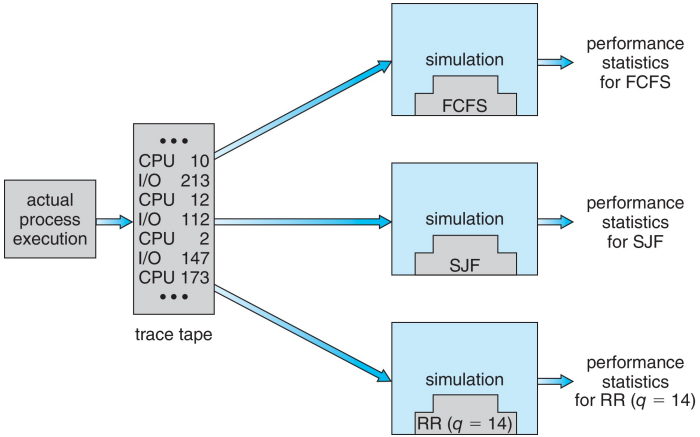
$$L_q = \lambda W_q$$

- L_q：等待队列中的平均任务数；W_q：平均排队时间。
- 若平均每秒到达 7 个任务，队列平均有 14 个任务，则 $W_q = 14/7 = 2$ 秒。
- 若统计包含正在服务的任务，应改用系统内平均数量与逗留时间。
- 定律不要求某一种特定调度算法或指数分布，但需要长期流量平衡。

资料：1. Operating System Concepts，第 5 章。

用仿真比较算法

把同一组到达、CPU 与 I/O 事件交给不同算法，分别统计性能指标。



资料：1. Operating System Concepts，第 5 章。

仿真的时钟、事件与输出

仿真用事件推进时间：任务到达、时间片到期或执行结束时，更新队列并重新选择任务。

部分	本章算法例子的做法
输入	到达时刻、CPU 执行时间、优先级、时间片
事件	任务到达、CPU 区间结束、时间片到期
时钟	从当前时刻推进到下一个相关事件
输出	执行时间线与各任务等待、周转、响应时间

还可以加入 I/O 完成、切换代价等事件，模拟更完整的执行过程。

资料：1. Operating System Concepts，第 5 章。

实际系统中的调度性能测量

在真实系统中运行相同的工作负载，观察调度算法连同实现开销的实际表现。

- 同一工作负载在不同硬件、版本和系统配置上，表现可能不同。
- 记录吞吐量、响应分布、CPU 时间、切换次数和实验环境。
- 实际更换内核调度器成本较高；参数或策略调整也会改变其他任务的行为。
- 控制后台负载与资源配额，使用相同条件重复测量。
- 先用可重复的基准观察行为，再判断实现改动是否达到了既定目标。

资料：1. Operating System Concepts，第 5 章。

本章小结

调度是在资源和时间约束下选择就绪任务，需要把规则与评价指标对应起来。

- 调度时机决定何时选择，抢占规则决定能否换下仍可运行的任务。
- 等待、周转和响应时间的口径不同，不能相互替代。
- 基本算法可组合成多级队列、反馈队列和实际系统策略。
- 多核还涉及放置与迁移，实时任务还涉及截止时间。
- 用相同工作负载计算、建模和测量，比较目标指标。

资料：1. Operating System Concepts，第 5 章。

课堂练习

- 某任务首次运行很早，是否意味着它的总等待时间一定很短?
- 新任务到达时，非抢占 SJF 与 SRTF 的选择有什么区别?
- RR 时间片到期，是否一定会切换到另一个进程?
- 四核八线程是否等于八个独立物理 CPU 核?
- Linux 的 nice、实时优先级与 EEVDF 虚拟截止时间，各表示什么?
- 同一算法在一个固定例题中表现最好，是否足以证明它适合所有工作负载?

参考资料

[1] Silberschatz, Galvin, Gagne (2018) —Operating System Concepts, 10th ed., Chapter 5: CPU Scheduling

[2] Linux man-pages —sched(7): overview of CPU scheduling

[3] Linux man-pages —pthread_attr_setscope(3): thread contention scope

[4] Linux man-pages —sched_setaffinity(2): set and get CPU affinity

[5] The Linux Kernel documentation —CFS Scheduler

[6] The Linux Kernel documentation —EEVDF Scheduler

[7] The Linux Kernel documentation —Scheduler Nice Design

[8] The Linux Kernel documentation —Deadline Task Scheduling

[9] Microsoft Learn —Scheduling Priorities

[10] Microsoft Learn —Priority Boosts

[11] procps-ng —ps(1): report a snapshot of the current processes

[12] util-linux —chrt(1): manipulate real-time attributes of a process

[13] util-linux —taskset(1): set or retrieve CPU affinity

[14] The Linux Kernel documentation —Control Group v2: CPU Interface Files

[15] C. L. Liu and James W. Layland (1973) —Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment, Journal of the ACM 20(1), 46–61

[16] Linux v6.12 —include/linux/sched/prio.h